LAB GUIDE · 32 LABS · SIX PARTS

Python Programming Hands-On Lab Guide

One lab per video. Each lab turns the lesson into something you build, then something you deliberately break and repair. Work through them in order — later labs reuse files, environments and habits from earlier ones.

How each lab works

01 Goal	02 Steps	03 Break it	04 Checkpoint
What you will be able to do when the lab is done.	Build it. Type the code yourself — don't paste.	Cause the lesson's typical failure on purpose, then fix it to root cause.	Don't move on until every box is true.

Before you start

- A computer running Windows, macOS or Linux, with permission to install software.
- Python 3.11 or newer. Commands in this guide use `python3`; on Windows, use `py` or `python`.
- A code editor with Python support (VS Code or PyCharm Community).
- One folder, `python-labs/`, with a subfolder per lab: `lab01/`, `lab02/`, and so on.
- A lab notebook — paper or a text file — for predictions, error logs and checkpoint answers.

Contents

LAB	TITLE	PART
01	Run Python Interactively	I · Foundations
02	A Project Environment That Works	I · Foundations
03	Typed Input and Safe Conversion	I · Foundations
04	A Calculator You Predict First	I · Foundations
05	Reading Tracebacks	I · Foundations

LAB	TITLE	PART
06	Multi-Branch Classifier	II · Control Flow
07	Loops, Patterns and Loop Bugs	II · Control Flow
08	Refactor a Script into Functions	II · Control Flow
09	Recursion and Higher-Order Functions	II · Control Flow
10	A Text-Processing Pipeline	II · Control Flow
11	Read, Transform, Write a File	II · Control Flow
12	List Manager and the Aliasing Trap	III · Data
13	Model an Entity as Nested Data	III · Data
14	Same Problem, Different Structures	III · Data
15	Clean Messy CSV into JSON	III · Data
16	Go Lazy with Generators	III · Data
17	Make a Fragile Program Robust	IV · Robust Programs
18	Split a Script into a Package	IV · Robust Programs
19	Your First Class	IV · Robust Programs
20	Inheritance, Special Methods, Composition	IV · Robust Programs
21	A Test Suite That Catches Regressions	IV · Robust Programs
22	Step Through a Bug, Then Log It	IV · Robust Programs
23	Evaluate and Pin a Library	V · Real World
24	A Safe Batch-File CLI	V · Real World
25	An API Client That Survives the Network	V · Real World
26	Answer a Question with Data	V · Real World
27	A Small Web Service with Storage	V · Real World
28	Profile, Optimize, Prove It	V · Real World
29	Refactor for Readability	VI · Practice

LAB	TITLE	PART
30	Capstone: End-to-End Application	VI · Practice
31	Version Control and Publishing	VI · Practice
32	Self-Assessment and 90-Day Plan	VI · Practice

PART I · LABS 01–05

Foundations

How code runs, a working environment, values and expressions — and the habit of reading errors instead of fearing them.

LAB 01 · VIDEO 1 30 MIN

Run Python Interactively

Goal. Run statements at the interactive prompt and from a saved file, and explain why they behave differently.

STEPS

1. Open a terminal and run `python3 --version`. Write the version in your notebook.
2. Start the interpreter with `python3`. At the >>> prompt, enter each line below and note what appears.
3. Exit with `exit()`. Save the same lines as `lab01/first.py` and run `python3 first.py`.
4. Compare the two outputs side by side.

```
2 + 3
"ha" * 3
print("Hello, interpreter")
name = "Ada"
name
```

BREAK IT

Add `print("unclosed` as line 2 of `first.py`. Run it. Note that *nothing* printed — not even line 1. Then type the same line at the prompt. Write down what this tells you about when Python reads versus runs a file.

CHECKPOINT

- You can explain why `2 + 3` shows 5 at the prompt but nothing in the file.
- You know when to use the prompt (quick experiments) and when to use a file (anything you keep).

LAB 02 · VIDEO 2 45 MIN

A Project Environment That Works

Goal. Create an isolated virtual environment, install a package into it, and prove which interpreter is running your code.

STEPS

1. In `lab02/`, create an environment: `python3 -m venv .venv`.
2. Activate it — macOS/Linux: `source .venv/bin/activate`; Windows: `.venv\Scripts\activate`. Your prompt should now show `(.venv)`.
3. Install a package: `python -m pip install rich`.
4. Open the folder in your editor and select `.venv` as the project interpreter.
5. Save and run `check_env.py`.

```
import sys
from rich import print

print(f"[bold]Interpreter:[/bold] {sys.executable}")
print(f"[bold]Version:[/bold] {sys.version.split()[0]}")
print("[green]Environment OK[/green]")
```

BREAK IT

Run `deactivate`, then run the script again. Read the `ModuleNotFoundError`. Use `python3 -c "import sys; print(sys.executable)"` to confirm the wrong interpreter ran. Reactivate and confirm the fix.

CHECKPOINT

- The script prints a path inside `lab02/.venv`.
- You can diagnose "installed but can't import" in under a minute.
- You always install with `python -m pip`, so pip matches the interpreter.

LAB 03 · VIDEO 3 40 MIN

Typed Input and Safe Conversion

Goal. Read text and numbers from the user, convert deliberately, and handle input that can't be converted.

STEPS

1. Write `profile.py` below. Run it with valid input, then with `twelve` as the age.
2. At the prompt, run `type(input("x: "))` and enter `5`. Note that input is always a string.

3. Run `0.1 + 0.2` and `0.1 + 0.2 == 0.3`. Then try `round(0.1 + 0.2, 2)`.
4. Try `int("3.5")`, `int(float("3.5"))`, `bool("False")`. Record each result and why.

```
name = input("Name: ").strip()
age_text = input("Age: ").strip()
height_m = float(input("Height in metres: "))

if age_text.isdigit():
    age = int(age_text)
    print(f"{name} turns {age + 1} next year.")
else:
    print(f"'{age_text}' isn't a whole number, so age was skipped.")

print(f"Height in cm: {height_m * 100:.1f}")
```

BREAK IT

Replace `age + 1` with `age_text + 1`. Read the `TypeError` and say in one sentence which two types collided.

CHECKPOINT

- Valid and invalid ages both finish without a crash.
- You can explain why `0.1 + 0.2 != 0.3` and when it matters (money, equality checks).
- You can describe a variable as a name bound to a value, not a box.

LAB 04 · VIDEO 4 40 MIN

A Calculator You Predict First

Goal. Test your mental model of evaluation by predicting every result before Python tells you.

STEPS

1. Copy the table into your notebook and fill in the Prediction column — no running code yet.
2. Evaluate each expression at the prompt and fill in Actual. Circle every miss.
3. For each miss, write the rule you got wrong (precedence, floor division, chaining).
4. Write `calc.py`: read two numbers and print the result of `+ - * / // % **` for them, one per line, with labels.

EXPRESSION	PREDICTION	ACTUAL
`7 / 2·7 // 2·-7 // 2`		
`7 % 3·-7 % 3`		
`2 ** 3 ** 2`		
`2 + 3 * 4·(2 + 3) * 4`		
`1 < 2 < 3·3 > 2 == True`		
`not True or True`		
`"ab" * 3 + "c"`		
`x = 5; x += 2; x *= 3; x`		

BREAK IT

In `calc.py`, compute an average as `a + b / 2`. Run it with 4 and 8; explain the wrong answer, then fix it with parentheses.

CHECKPOINT

- Every circled miss has a written rule beside it.
- You use parentheses whenever intent might be unclear to a reader.

LAB 05 · VIDEO 5 60 MIN

Reading Tracebacks

Goal. Diagnose broken programs from the traceback alone, fixing each to root cause with one change at a time.

STEPS

1. Save each snippet below as its own file (`bug1.py` … `bug5.py`) and run it.
2. Read each traceback from the bottom up: error type, message, then the file and line.
3. Before editing, write a hypothesis in your notebook: "It fails because…".
4. Make one change, rerun, confirm. If it didn't work, revert before trying the next idea.
5. Where the cause isn't obvious, add a `print()` of the values involved just above the failing line.

```
# bug1.py
if 3 > 2
    print("yes")

# bug2.py
total = 10
print(totl)

# bug3.py
age = 30
print("Age: " + age)

# bug4.py
count = 0
print(100 / count)

# bug5.py
items = ["a", "b", "c"]
print(items[3])
```

FILE	ERROR TYPE	LINE	ROOT CAUSE	FIX
bug1				
bug2				
bug3				
bug4				
bug5				

BREAK IT

Trade a working file with a partner (or ask an AI assistant to plant one bug for you). Diagnose it using only the traceback — no reading the code top to bottom first.

CHECKPOINT

- The table is complete, and every fix is a single change.
- You can name what `SyntaxError`, `NameError`, `TypeError`, `ZeroDivisionError` and `IndexError` each mean.

PART II · LABS 06–11

Control Flow, Functions and I/O

Decisions, repetition, reusable functions, text and files — everything a script needs to do real work.

LAB 06 · VIDEO 6 40 MIN

Multi-Branch Classifier

Goal. Classify input with `if / elif / else`, and find a logic error that gives wrong output without crashing.

STEPS

1. Type in `grade.py` exactly as shown, including the bug.
2. Test it with 95, 85, 72, 65, 40, 100 and 0. Record each output.
3. Find why 95 gets the wrong letter. Fix it by reordering the branches.
4. Add validation: scores below 0 or above 100 print `invalid`.
5. At the prompt, test truthiness: `bool("")`, `bool("0")`, `bool(0)`, `bool([])`, `bool(None)`.

```
score = int(input("Score: "))

if score >= 60:
    letter = "D"
elif score >= 70:
    letter = "C"
elif score >= 80:
    letter = "B"
elif score >= 90:
    letter = "A"
else:
    letter = "F"

print(f"{score} → {letter}")
```

BREAK IT

Add `name = input("Name: ")` and `if name: print("Hi", name)`. Press Enter without typing. Explain why the branch didn't run — then write a condition that's always true by mistake (`if letter == "A" or "B":`) and explain that too.

CHECKPOINT

- All seven test scores, plus the boundary values 59, 60, 89 and 90, give the right letter.
- You know why a logic error is harder to find than a crash: there's no traceback.

LAB 07 · VIDEO 7 45 MIN

Loops, Patterns and Loop Bugs

Goal. Process a collection with `for`, repeat with `while`, and cause and fix the two classic loop bugs.

STEPS

1. Given `temps = [18, 22, 25, 31, 15, 28]`, use one `for` loop to find the total, the max and how many days were above 24.
2. Use `continue` to skip negative readings and `break` to stop at the first reading over 40.
3. Print a right triangle of `*` with 5 rows using nested loops, then an inverted one.
4. Write a `while` countdown from 10 to 1 followed by `Liftoff`.

```
for row in range(1, 6):
    print("*" * row)

n = 10
while n > 0:
    print(n)
    n -= 1
print("Liftoff")
```

BREAK IT

Infinite loop: delete `n -= 1`, run it, stop it with Ctrl+C, and explain why the condition never became false. **Off-by-one:** try to print 1 to 10 with `range(1, 10)`; count the output and fix the bound.

CHECKPOINT

- Every `while` loop you write has a line that moves it toward stopping.
- You can say how many times `range(a, b)` runs without running it.
- You know nested loops over *n* items do roughly *n* × *n* work.

LAB 08 · VIDEO 8 50 MIN

Refactor a Script into Functions

Goal. Turn a long script into small, single-purpose functions without changing what it outputs.

STEPS

1. Save `before.py` below. Capture its output: `python3 before.py > before.txt`.
2. Copy it to `after.py`. Extract `parse_scores(text)`, `average(nums)`, `grade_band(avg)` and `report(name, nums)`. Each should do one thing and `return` a value.
3. Give `report` a keyword argument `decimals=1`.
4. Capture `after.txt` and compare: `diff before.txt after.txt` (Windows: `fc`). No output means identical.

```
data = "Ana:88,92,79;Ben:65,70,58;Cy:95,98,100"
for chunk in data.split(";"):
    name, raw = chunk.split(":")
    nums = [int(x) for x in raw.split(",")]
    total = 0
    for n in nums:
        total += n
    avg = total / len(nums)
    if avg >= 90:
        band = "high"
    elif avg >= 70:
        band = "mid"
    else:
        band = "low"
    print(f"{name}: avg {avg:.1f} ({band})")
```

BREAK IT

Remove the `return` from `average` and run — explain the `None`. Then set a variable inside a function and try to print it outside; read the `NameError` and explain local scope.

CHECKPOINT

- `diff` shows no differences.
- Each function name describes everything it does — no "and" needed.

LAB 09 · VIDEO 9 50 MIN

Recursion and Higher-Order Functions

Goal. Write a recursive function with a correct base case, and a function that takes another function as an argument.

STEPS

1. Write `deep_sum` to total a nested list such as `[1, [2, 3], [4, [5, 6]]]`. Add a docstring.

2. Write `apply_to_all(func, items)` that returns a new list. Call it with `str.upper`, with your own named function, and with a `lambda`.
3. Write `total(*nums)` that accepts any number of arguments.
4. Use `sorted(words, key=len)` and explain what is being passed as `key`.

```
def deep_sum(items):
    """Return the sum of all numbers in a nested list."""
    total = 0
    for item in items:
        if isinstance(item, list):
            total += deep_sum(item)   # recursive case
        else:
            total += item             # base case: a plain number
    return total

def apply_to_all(func, items):
    return [func(x) for x in items]

print(apply_to_all(lambda x: x * 2, [1, 2, 3]))
```

BREAK IT

Write `def countdown(n): print(n); countdown(n - 1)` with no base case and read the `RecursionError`. Then write `def add_item(x, bag=[]): bag.append(x); return bag`, call it three times, and explain the growing list. Fix it with `bag=None`.

CHECKPOINT

- You can point to the base case in every recursive function you write.
- You can explain why a pure function is easier to test than one that changes outside state.

LAB 10 · VIDEO 10 50 MIN

A Text-Processing Pipeline

Goal. Parse, transform and reformat text, including non-ASCII characters, with string methods and f-strings.

STEPS

1. Start with the raw contacts below. Each line is `last, first <email>` with inconsistent spacing and case.
2. For each line: split on the comma, `.strip()` parts, title-case the names and lower-case the email.
3. Print an aligned table using f-string width specifiers, e.g. `f"{full:<20}{email}"`.

4. Print each person's initials using indexing, and the email domain using slicing or `.split("@")`.
5. Encode `"Zoë"` to bytes with `.encode("utf-8")`, print the bytes, then decode back.

```
raw = """lovelace,  ada <ADA@Example.com>
  hopper , GRACE <grace@navy.MIL>
müller, zoë <zoe@beispiel.de>"""

for line in raw.splitlines():
    names, email = line.split("<")
    last, first = [p.strip().title() for p in names.split(",")]
    email = email.strip(" >").lower()
    print(f"{first + ' ' + last:<20}{email}")
```

BREAK IT

Try `name = "ada"; name[0] = "A"` and explain the immutability error. Then run `"Zoë".encode("utf-8").decode("ascii")`, read the `UnicodeDecodeError`, and fix it by decoding with the encoding that was used to encode.

CHECKPOINT

- "Müller" and "Zoë" print correctly and line up in the table.
- You can explain the difference between `str` (text) and `bytes` (encoded data).

LAB 11 · VIDEO 11 45 MIN

Read, Transform, Write a File

Goal. Read a file safely, transform its contents line by line, write the result to a new file, and handle a missing file gracefully.

STEPS

1. Create `notes.txt` with at least six lines, including a few blank ones.
2. Write `number_lines.py`: skip blank lines, number the rest, write to `notes_numbered.txt`.
3. Take the input file name from `sys.argv`, with `notes.txt` as the default.
4. Always pass `encoding="utf-8"` and always open files with `with`.

```
import sys
from pathlib import Path

src = Path(sys.argv[1] if len(sys.argv) > 1 else "notes.txt")
if not src.exists():
    print(f"Can't find {src.resolve()} — check the name and folder.")
    sys.exit(1)

dst = src.with_name(src.stem + "_numbered.txt")
with open(src, encoding="utf-8") as fin, open(dst, "w", encoding="utf-8") as fout:
    n = 0
    for line in fin:              # one line at a time
        if line.strip():
            n += 1
            fout.write(f"{n:03}  {line}")
print(f"Wrote {n} lines to {dst}")
```

BREAK IT

Run with `nothere.txt` and confirm the friendly message. Run from a different folder (`cd ..`) and explain why the relative path failed. On macOS/Linux, `chmod 000 notes.txt` and read the permission error, then restore with `chmod 644`.

CHECKPOINT

- The output file exists and has no blank lines.
- You can explain why iterating line by line scales to files bigger than memory and `.read()` doesn't.

PART III · LABS 12–16

Data Structures and Data

Lists, dictionaries, tuples and sets; the formats programs exchange; and processing data lazily, one item at a time.

LAB 12 · VIDEO 12 45 MIN

List Manager and the Aliasing Trap

Goal. Manage a list with its methods and slicing, then trigger and fix the bug where two names point at one list.

STEPS

1. Build a menu loop with commands add, `remove`, `show`, `sort`, `top3` and `quit` for a shopping list.
2. `show` prints items numbered from 1 using `enumerate(items, start=1)`.
3. `top3` uses a slice; `remove` checks `in` before removing.
4. Add undo: save a snapshot before each change and restore it.

```
items = ["milk", "eggs"]
snapshot = items          # the bug: same list, two names
items.append("bread")
print(snapshot)           # ['milk', 'eggs', 'bread'] — undo is broken
print(snapshot is items)  # True

snapshot = items.copy()   # the fix: a new list
```

BREAK IT

Implement undo with `snapshot = items` first and watch it fail. Fix with `.copy()`. Then try removing every item that starts with "e" inside `for item in items:` and explain the skipped item; fix by looping over `items[:]` or building a new list. Finally, request `items[99]` and read the `IndexError`.

CHECKPOINT

- Undo restores the previous list exactly.
- You can predict the output of `a = [1]; b = a; b.append(2); a`.

LAB 13 · VIDEO 13 45 MIN

Model an Entity as Nested Data

Goal. Represent a real-world entity with nested dictionaries and lists, then navigate and update it safely.

STEPS

1. Model a library member: name, ID, contact details (a dict), and loans (a list of dicts with title, due date, renewals).
2. Write functions to print all loan titles, add a loan, and renew a loan by title.
3. Read optional fields with `.get()` and a default — the member may have no phone.
4. Loop with `.items()` to print every top-level field as `key: value`.
5. Write a short note on why loans are a list but the member is a dict.

```
member = {
    "id": "M-1042",
    "name": "Ada Lovelace",
    "contact": {"email": "ada@example.com"},
    "loans": [
        {"title": "Dune", "due": "2026-11-02", "renewals": 0},
        {"title": "SICP", "due": "2026-11-09", "renewals": 1},
    ],
}
phone = member["contact"].get("phone", "not on file")
member["loans"][0]["renewals"] += 1
```

BREAK IT

Access `member["contact"]["phone"]` and read the `KeyError`. Then try using a list as a key — `{["a", "b"]: 1}` — and explain the `TypeError: unhashable type`. Fix it with a tuple.

CHECKPOINT

- Renewing a title that isn't on loan prints a clear message instead of crashing.
- You choose a dict when you look things up by name, and a list when order and position matter.

LAB 14 · VIDEO 14 45 MIN

Same Problem, Different Structures

Goal. Measure how the choice of structure changes speed, and rewrite a loop as a comprehension.

STEPS

1. Problem: check 1,000 user IDs against a list of 200,000 banned IDs. Time it with a list, then a set.
2. Record both times and the speedup in your notebook.

3. Store map coordinates as tuples, and use them as dictionary keys.
4. Use `&`, `|` and `-` to find students in both clubs, either club, and only the first.
5. Rewrite the loop below as a comprehension and confirm both produce equal lists.

```
import random, time

banned_list = list(range(0, 400_000, 2))
banned_set = set(banned_list)
checks = [random.randrange(400_000) for _ in range(1_000)]

for label, banned in [("list", banned_list), ("set", banned_set)]:
    start = time.perf_counter()
    hits = sum(1 for c in checks if c in banned)
    print(f"{label}: {time.perf_counter() - start:.4f}s, {hits} hits")

# Rewrite as a comprehension:
squares_of_evens = []
for n in range(20):
    if n % 2 == 0:
        squares_of_evens.append(n * n)
```

BREAK IT

Try `point = (3, 4); point[0] = 5`. Write down why that error is a *feature* for a coordinate that should never change.

CHECKPOINT

- You've recorded a measured speedup for set membership.
- For a given dataset, you can justify list, dict, tuple or set in one sentence.

LAB 15 · VIDEO 15 60 MIN

Clean Messy CSV into JSON

Goal. Read untrustworthy tabular data, validate and clean each row, and write the clean records as JSON plus a log of rejects.

STEPS

1. Save `orders.csv` below as UTF-8.
2. Read it with `csv.DictReader`. For each row, strip whitespace, normalize the email, and convert quantity to `int` and price to `float`.

3. Reject rows with a missing ID, a non-numeric value or a negative quantity. Record the row number and reason.
4. Write clean rows to `orders.json` with `json.dump(..., indent=2, ensure_ascii=False)` and rejects to `rejects.csv`.
5. Load `orders.json` back and check the record count matches.

```
id,email,qty,price
101, Ana@Example.com ,2,9.99
102,ben@example.com,,4.50
,cy@example.com,1,3.00
104,dee@example.com,-3,7.25
105,  José@Example.com,1,"12.00"
106,eve@example.com,two,5.00
```

BREAK IT

Remove your validation and run again — note exactly which row crashes the program. Then open the CSV without `encoding="utf-8"` on Windows (or save it as Latin-1) and watch "José" garble or fail.

CHECKPOINT

- Two clean records, four rejects, each with a reason.
- Your program never assumes a field exists or has the right type.

LAB 16 · VIDEO 16 50 MIN

Go Lazy with Generators

Goal. Rewrite a program that loads everything into memory so it streams data through generators instead.

STEPS

1. Generate a large log: write 5,000,000 lines like `2026-10-20 ERROR disk full`, mixing levels randomly.
2. Eager version: read all lines into a list, filter ERRORs into a second list, count them. Measure peak memory with `tracemalloc`.
3. Lazy version: chain generators as below. Measure again.
4. Compare `sum([x*x for x in range(10**7)])` with `sum(x*x for x in range(10**7))`.
5. Use `itertools.islice` to take the first 5 errors without reading the whole file.

```
import tracemalloc

def read_lines(path):
    with open(path, encoding="utf-8") as f:
        for line in f:
            yield line.rstrip("\n")

def only(level, lines):
    return (ln for ln in lines if f" {level} " in ln)

tracemalloc.start()
count = sum(1 for _ in only("ERROR", read_lines("big.log")))
_, peak = tracemalloc.get_traced_memory()
print(f"{count:,} errors, peak {peak / 1e6:.1f} MB")
```

BREAK IT

Assign `errs = only("ERROR", read_lines("big.log"))`, call `sum(1 for _ in errs)` twice, and explain the second result of 0. Then write `itertools.count()` in a `for` loop with no `break` and stop it with Ctrl+C.

CHECKPOINT

- The lazy version's peak memory stays small and flat however big the file gets.
- You can say when eager is fine (small data, reused many times) and when lazy wins.

PART IV · LABS 17–22

Robust, Organized Programs

Handling failure, structuring code across files, modeling with objects, and proving correctness with tests and diagnostics.

LAB 17 · VIDEO 17 50 MIN

Make a Fragile Program Robust

Goal. Handle each realistic failure with the right exception at the right level, without hiding genuine bugs.

STEPS

1. Save `fragile.py`. List every way it can fail: no file, bad JSON, missing key, non-numeric value, zero people.
2. Catch `FileNotFoundError` and `json.JSONDecodeError` where the file is loaded, with a helpful message.
3. In `split_bill`, `raise ValueError` for zero or negative people — don't print from inside it.
4. Add a `finally` block that prints done on success and failure alike.
5. Run each failure case and confirm the message is specific.

```
# fragile.py — reads bill.json: {"total": 120.0, "people": 4}
import json

def split_bill(total, people):
    return total / people

with open("bill.json") as f:
    bill = json.load(f)
print(f"Each pays {split_bill(bill['total'], bill['people']):.2f}")
```

BREAK IT

Wrap the whole program in `try: … except: print("Something went wrong")`. Introduce a typo (`bill['totl']`) and note how the bare except hides it. Replace it with specific exceptions and watch the real bug surface.

CHECKPOINT

- Every `except` names a specific exception type.

- Expected failures get friendly messages; programming errors still produce a traceback.

LAB 18 · VIDEO 18 50 MIN

Split a Script into a Package

Goal. Reorganize a single-file program into a package of focused modules that still runs the same way.

STEPS

1. Start from your Lab 12 list manager (or Lab 15 cleaner). Capture its output for a fixed set of inputs.
2. Create the layout below. Move data handling to `storage.py`, logic to `core.py`, and the menu to `cli.py`.
3. Use explicit imports: `from inventory.core import add_item`. Avoid `import *`.
4. Add `__main__.py` so `python -m inventory` starts the program.
5. Use one standard-library module you haven't used before (e.g. `datetime` for timestamps).

```
lab18/
├── inventory/
│   ├── __init__.py
│   ├── __main__.py     # from inventory.cli import main; main()
│   ├── cli.py          # input/output only
│   ├── core.py         # pure logic, no print()
│   └── storage.py      # load/save
└── README.md
```

BREAK IT

Run `python inventory/cli.py` directly and read the `ModuleNotFoundError`; print `sys.path` to see why. Then make `core.py` import from `storage.py` and `storage.py` import from `core.py` at the top; read the circular import error and fix it by moving the shared piece.

CHECKPOINT

- `python -m inventory` produces the same output as the original script.
- Only `cli.py` calls `print()` or `input()`.

LAB 19 · VIDEO 19 45 MIN

Your First Class

Goal. Model a bank account as a class, create several instances, and confirm each keeps its own state.

STEPS

1. Write `BankAccount` with `owner`, `balance` and a `history` list set in `__init__`.
2. Add `deposit`, `withdraw` (raise `ValueError` on overdraft) and `statement`.
3. Create three accounts, make different transactions on each, and print every statement.
4. Write one sentence each: what is the class, what is an instance, and what is `self` during `a.deposit(50)`?

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.history = []

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.balance += amount
        self.history.append(("deposit", amount))

a = BankAccount("Ana")
b = BankAccount("Ben", 100)
a.deposit(50)
print(a.balance, b.balance)   # 50 100
```

BREAK IT

Remove `self` from `def deposit(self, amount)` and read the error. Restore it. Then move `history = []` to class level (outside `__init__`), deposit into two accounts, and watch both histories show both deposits.

CHECKPOINT

- Three accounts, three independent balances and histories.
- You can name one problem in this course where a plain function would have been better than a class.

LAB 20 · VIDEO 20 60 MIN

Inheritance, Special Methods, Composition

Goal. Extend `BankAccount` through inheritance and special methods, then untangle a bad hierarchy using composition.

STEPS

1. Create `SavingsAccount(BankAccount)` with an interest rate and `add_interest()`. Override `withdraw` to allow at most three withdrawals a month, calling `super().withdraw()`.
2. Add `__repr__`, `__eq__` (same owner and balance) and `__lt__` (by balance), then `sorted()` a list of accounts.
3. Rename `balance` to `_balance` and expose it with a read-only `@property`.
4. Refactor the tangle below: replace the three subclasses with one `BankAccount` that *has* an optional `notifier` and `auditor`.

```
# The tangle — every combination needs a new subclass
class EmailAccount(BankAccount): ...
class AuditedAccount(BankAccount): ...
class EmailAuditedAccount(EmailAccount, AuditedAccount): ...

# The goal — behavior plugged in from parts
acct = BankAccount("Ana", notifier=EmailNotifier(), auditor=AuditLog())
```

BREAK IT

Outside the class, set `acct._balance = 1_000_000`. Note that Python allows it, and write down what the underscore promises and what it doesn't enforce.

CHECKPOINT

- `print(acct)` shows something useful, and `sorted(accounts)` works.
- Adding an SMS notifier needs a new small class, not a new subclass of account.

LAB 21 · VIDEO 21 60 MIN

A Test Suite That Catches Regressions

Goal. Write focused tests including edge cases, then watch them catch a bug you introduce.

STEPS

1. In a fresh venv, `python -m pip install pytest`.
2. Write `prices.py` with `parse_price(text)` that turns `"$1,299.99"` into `1299.99`.
3. Before finishing it, list edge cases in `test_prices.py`: no symbol, spaces, no decimals, zero, empty string, letters, negative.
4. Write each test as arrange–act–assert. Use `pytest.raises(ValueError)` for invalid input.
5. Run `pytest -v` until everything passes.

```
import pytest
from prices import parse_price

def test_parses_thousands_separator():
    text = "$1,299.99"              # arrange
    result = parse_price(text)      # act
    assert result == 1299.99        # assert

@pytest.mark.parametrize("bad", ["", "abc", "$", "1.2.3"])
def test_rejects_invalid(bad):
    with pytest.raises(ValueError):
        parse_price(bad)
```

BREAK IT

"Improve" `parse_price` by replacing the comma removal with `text.replace(",", ".")`. Run the suite and read which test catches the regression. Then write a test with no `assert` and note that it passes — a test that tests nothing.

CHECKPOINT

- At least eight tests, all passing, at least four of them edge cases.
- Your introduced regression turned at least one test red.

LAB 22 · VIDEO 22 50 MIN

Step Through a Bug, Then Log It

Goal. Find a non-obvious bug by stepping through code in the debugger, then add logging that would have revealed it faster.

STEPS

1. Run `stats.py`. The average should be 82.5, but it isn't.
2. Put `breakpoint()` at the top of `clean`. Use `n` (next), `s` (step), `p value` (print) and `c` (continue). Or use your editor's debugger with a breakpoint.
3. Watch `value` and `kept` on each pass until you see the moment it goes wrong. Write down the root cause.
4. Fix it. Then replace prints with `logging`: `DEBUG` for each skipped value, `INFO` for the summary, `WARNING` for unparseable input.
5. Run at `INFO` and `DEBUG` levels and compare the noise.

```
import logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(name)s: %(message)s")
log = logging.getLogger("stats")

def clean(raw):
    kept = []
    for value in raw:
        if value:                  # bug hides here
            kept.append(float(value))
    return kept

scores = clean(["90", "0", "", "95", "65", "n/a"])
print(sum(scores) / len(scores))
```

BREAK IT

Fix only the symptom first — catch the `ValueError` from `"n/a"` and move on. Note that the average is still wrong, then explain why a symptom fix isn't a root-cause fix. Hint: is the bug in the code or the data?

CHECKPOINT

- You found the bug by watching state change, not by guessing.
- The log at `DEBUG` level would have shown the problem on the first run.

PART V · LABS 23-28

Python in the Real World

The ecosystem, automation, the web, data analysis, applications and performance — applied with judgment and safety.

LAB 23 · VIDEO 23 45 MIN

Evaluate and Pin a Library

Goal. Choose a third-party library on evidence, integrate it, and pin it so the project installs the same everywhere.

STEPS

1. Problem: parse human-written dates ("next Friday", "20 Oct 2026", "2026-10-20T09:00"). First check what `datetime` alone can do.
2. Find two candidate libraries on PyPI. Score each on the checklist below.
3. Install the winner in a fresh venv, solve the problem using its documentation only.
4. Pin it: `python -m pip freeze > requirements.txt`. Review the file and remove anything unrelated.
5. Delete the venv, recreate it, run `python -m pip install -r requirements.txt`, and rerun your script.

CRITERION	CANDIDATE A	CANDIDATE B
Last release within 12 months		
More than one active maintainer		
Open issues answered		
Clear docs with examples		
License fits your use		
Dependencies it pulls in		
Exact package name verified (no typosquat)		

BREAK IT

Install an older major version of your library (`pip install "name==X.*"`) and run your script. Record what broke — that's what an unpinned dependency does on someone else's machine.

CHECKPOINT

- A completed scorecard justifies your choice.
- A fresh environment rebuilt from `requirements.txt` runs the script unchanged.

LAB 24 · VIDEO 24 60 MIN

A Safe Batch-File CLI

Goal. Build a command-line tool that organizes files in bulk, with arguments, a dry-run default and guards against destructive mistakes.

STEPS

1. Make a sandbox: a script that creates `sandbox/` with 50 dummy files of mixed extensions. Never test on real folders.
2. Write `tidy.py` with `argparse`: a required `folder`, `--apply` (default is dry run), and `--limit` (default 100).
3. Move each file into a subfolder named by its extension. In dry run, print what *would* happen.
4. Guards: refuse to run on your home folder or a drive root; stop if the file count exceeds `--limit`; never overwrite an existing file.
5. Catch per-file errors, log them, and continue; print a summary at the end.
6. Optional: schedule it with cron (macOS/Linux) or Task Scheduler (Windows) against the sandbox.

```
import argparse
from pathlib import Path

p = argparse.ArgumentParser(description="Sort files into folders by extension.")
p.add_argument("folder", type=Path)
p.add_argument("--apply", action="store_true", help="actually move files")
p.add_argument("--limit", type=int, default=100)
args = p.parse_args()

target = args.folder.resolve()
if target in (Path.home(), Path(target.anchor)):
    raise SystemExit(f"Refusing to run on {target}")
```

BREAK IT

In the sandbox, make one file read-only (or lock it open on Windows) and run with `--apply`. Confirm the batch finishes and reports the one failure, rather than stopping halfway.

CHECKPOINT

- `python tidy.py --help` explains the tool.
- A run without `--apply` changes nothing on disk.
- Every guard has been triggered at least once on purpose.

LAB 25 · VIDEO 25 60 MIN

An API Client That Survives the Network

Goal. Authenticate to a real API, fetch and parse JSON, and handle timeouts, errors and rate limits gracefully.

STEPS

1. Pick an API with free token auth (e.g. GitHub). Read its docs for auth, rate limits and terms of use.
2. Create a token and store it in an environment variable — never in code. Add `.env` to `.gitignore` if you use one.
3. Install `requests`. Fetch a list resource (e.g. your repositories) with a timeout on every call.
4. Validate the response shape before using it; print a short table of name, stars and last update.
5. On HTTP 429 or 403 with a rate-limit header, wait and retry with exponential backoff, at most 3 times.

```
import os, time, requests

TOKEN = os.environ["GITHUB_TOKEN"]
HEADERS = {"Authorization": f"Bearer {TOKEN}", "Accept":
"application/vnd.github+json"}

def get_json(url, tries=3):
    for attempt in range(tries):
        try:
            r = requests.get(url, headers=HEADERS, timeout=10)
        except requests.Timeout:
            print(f"Timeout, retry {attempt + 1}"); continue
        if r.status_code in (403, 429):
            wait = int(r.headers.get("Retry-After", 2 ** attempt))
            time.sleep(wait); continue
        r.raise_for_status()
        return r.json()
    raise RuntimeError(f"Gave up on {url}")
```

BREAK IT

Set `timeout=0.001` and confirm the retry path runs. Use a bad token and read the 401. Request a URL that returns an object instead of a list and confirm your shape check catches it.

CHECKPOINT

- `grep -r "ghp_" .` (or a search in your editor) finds no token in your files.
- Every request has a timeout; every failure path ends in a clear message.

LAB 26 · VIDEO 26 75 MIN

Answer a Question with Data

Goal. Load a real dataset, clean it, answer one question by transforming and aggregating, and chart the result honestly.

STEPS

1. Install `pandas`, `matplotlib` and `jupyterlab` (or use a plain script).
2. Choose a public CSV — city weather, bike-share trips, or a government open-data portal. Write your question first, e.g. "Which month has the most rides?"
3. Explore: `df.shape`, `df.info()`, `df.head()`, `df.isna().sum()`, `df.describe()`.
4. Clean: fix types (`pd.to_datetime`), drop or fill missing values, remove duplicates. Record every row you drop and why.

5. Answer with `groupby` and an aggregation. Sanity-check one number by hand.
6. Plot it with labelled axes, a title stating the finding, and a y-axis starting at zero for bar charts. Save as PNG.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("trips.csv", parse_dates=["start_time"])
before = len(df)
df = df.dropna(subset=["start_time"]).drop_duplicates()
print(f"Dropped {before - len(df)} rows")

by_month = df.groupby(df["start_time"].dt.month).size()
ax = by_month.plot(kind="bar", title="Rides peak in July")
ax.set_xlabel("Month"); ax.set_ylabel("Rides")
plt.tight_layout(); plt.savefig("rides_by_month.png", dpi=150)
```

BREAK IT

Skip the cleaning step and compute the answer again. Then replot with the y-axis starting just below the minimum. Write two sentences on how each change could mislead a reader.

CHECKPOINT

- A one-paragraph answer, the chart, and a list of cleaning decisions.
- One result has been checked independently of pandas.

LAB 27 · VIDEO 27 75 MIN

A Small Web Service with Storage

Goal. Build a complete notes service that stores data in a database and serves it over HTTP, with data, logic and interface kept separate.

STEPS

1. Install `flask`. Create db.py (SQLite via the standard `sqlite3` module), `notes.py` (validation and logic) and app.py (routes only).
2. Endpoints: `GET /notes`, `GET /notes/<id>`, `POST /notes` with JSON `title` and body.
3. Return 400 for invalid input and 404 for a missing note, each with a JSON error message.
4. Always use parameterized SQL (`?` placeholders) — never f-strings.
5. Run with `flask --app app run` and exercise it with `curl` or your browser.

```
curl -X POST localhost:5000/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "First", "body": "Stored in SQLite"}'

curl localhost:5000/notes
curl localhost:5000/notes/999     # expect 404
```

BREAK IT

Write a 20-thread script that POSTs notes at the same time. Check that all 20 were stored. If you shared one database connection across requests, watch it fail, and fix it with a connection per request.

CHECKPOINT

- Notes survive a server restart.
- app.py contains no SQL; db.py contains no Flask.

LAB 28 · VIDEO 28 50 MIN

Profile, Optimize, Prove It

Goal. Find a slow program's real bottleneck by measurement, fix it, and confirm the speedup — then use concurrency where waiting dominates.

STEPS

1. Before profiling, write down which function you *think* is slow.
2. Run `python -m cProfile -s cumtime slow.py`. Find the function with the highest cumulative time.
3. Fix the bottleneck with a better data structure or algorithm. Time both versions with `time.perf_counter`; confirm outputs match.
4. Concurrency: simulate 20 network calls with `time.sleep(0.5)`. Run them in sequence, then with `concurrent.futures.ThreadPoolExecutor`. Compare times.

```
# slow.py
import random

def load():
    return [random.randrange(50_000) for _ in range(20_000)]

def find_duplicates(nums):
    dups = []
    for i, a in enumerate(nums):
        if a in nums[i + 1:] and a not in dups:
            dups.append(a)
    return dups

def format_report(dups):
    return ", ".join(str(d) for d in sorted(dups))

print(len(format_report(find_duplicates(load()))))
```

BREAK IT

"Optimize" `format_report` first and measure. Record how little it changed. Then try the thread pool on a CPU-heavy task (summing squares) and explain why it didn't speed up.

CHECKPOINT

- A before/after table: your guess, the profiler's answer, both timings.
- You can say when threads help (waiting on I/O) and when they don't (pure computation).

PART VI · LABS 29-32

Professional Practice and Mastery

Code others can live with, a complete project, collaboration, and a plan for what comes next.

LAB 29 · VIDEO 29 60 MIN

Refactor for Readability

Goal. Make a working but ugly program readable, using tests to prove the behavior hasn't changed.

STEPS

1. Start with the code below. Write tests that pin its current behavior *before* changing anything (Lab 21).
2. Rename everything so the code reads like a description: `d` → `orders`, `f` → `total_by_customer`.
3. Remove duplication, extract functions, and replace magic numbers with named constants.
4. Run `ruff check` and `ruff format` (or `black`) to apply PEP 8 style.
5. Run the tests after every small change. Stop when a new reader would understand it in one pass.

```
def f(d):
    r = {}
    for x in d:
        if x[2] > 100: t = x[1] * x[2] * 0.9
        else: t = x[1] * x[2]
        if x[0] in r: r[x[0]] = r[x[0]] + t
        else: r[x[0]] = t
    return r
```

BREAK IT

Mid-refactor, change the discount rule in only one of two copies of the logic. Watch a test catch the inconsistency — the classic duplication bug.

CHECKPOINT

- All tests pass before and after; the linter reports nothing.
- No name in the file misdescribes what it holds or does.

LAB 30 · VIDEO 30 6-10 HRS

Capstone: End-to-End Application

Goal. Build a complete, tested, well-structured application from a problem statement to a package someone else can install.

STEPS

1. **Problem.** Write half a page: who it's for, what it does, what it won't do. Example: a personal expense tracker that imports bank CSVs and reports monthly spending by category.
2. **Design.** Sketch modules, data structures and the file or database format before coding.
3. **Build incrementally.** One feature at a time, each with tests, each committed (Lab 31) when green.
4. **Use the ecosystem.** Pull in at most two evaluated, pinned libraries (Lab 23).
5. **Harden.** Feed it malformed input, missing files and empty data. Handle each deliberately (Labs 15, 17).
6. **Polish.** Refactor (Lab 29), add logging (Lab 22), write a README with install and usage.
7. **Package.** Add a py`project.toml` with a console-script entry point. Install it with `pip install .` in a brand-new venv and run it by name.

```
[project]
name = "spendtrack"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = []

[project.scripts]
spendtrack = "spendtrack.cli:main"
```

BREAK IT

Give the package to someone else (or a clean virtual machine). Watch them install it using only your README. Every question they ask is a bug in your docs.

CHECKPOINT

- The test suite passes and covers the edge cases you found while hardening.
- A fresh install runs from the command line by name.
- The README is enough for a stranger to use it.

LAB 31 · VIDEO 31 60 MIN

Version Control and Publishing

Goal. Put a project under Git, work across branches, resolve a conflict, and publish it so others could use and contribute.

STEPS

1. In your capstone folder: `git init`. Write a `.gitignore` covering `.venv/`, `__pycache__/` and `.env` *before* the first commit.
2. Commit in small, meaningful steps with messages that say why.
3. Create `feature/export` and add a feature there. Meanwhile, on `main`, edit the same line of the README.
4. Merge the branch, resolve the conflict by hand, rerun tests, commit.
5. Add a LICENSE and a CONTRIBUTING note. Push to a hosted remote (GitHub, GitLab or Codeberg).
6. Read one file in an open-source Python project you use and write three things you learned.

```
git switch -c feature/export
# …edit, test…
git add -p                      # review each change as you stage it
git commit -m "Add CSV export so reports can open in spreadsheets"
git switch main
git merge feature/export        # resolve conflicts, then:
git add README.md && git commit
```

BREAK IT

In a throwaway repo, commit a file containing `API_KEY = "test-123"`, then delete it in a second commit. Run `git log -p` and find the key still in history. Write down why a leaked real secret must be revoked, not just deleted.

CHECKPOINT

- The public repo has a README, a license, passing tests and no secrets.
- `git log --oneline` tells the project's story.

LAB 32 · VIDEO 32 60 MIN

Self-Assessment and 90-Day Plan

Goal. Assess your skills honestly against the whole course, choose a direction, and commit to a concrete plan.

STEPS

1. Rate yourself 1–4 on each part: **1** need the notes, **2** can do with lookups, **3** can do unaided, **4** can teach it. Cite a lab as evidence.
2. Pick one specialization: automation, web development, data science, machine learning or DevOps.
3. Choose one project in that area slightly beyond your current level.
4. Fill in the 90-day plan with weekly hours you can actually keep.

PART	RATING	EVIDENCE (LAB)
I · Foundations		
II · Control flow, functions, I/O		
III · Data structures and data		
IV · Robust, organized programs		
V · Python in the real world		
VI · Professional practice		

PHASE	FOCUS	DELIVERABLE
Days 1–30	Close your weakest-rated part; learn your specialization's core library	
Days 31–60	Build the project's first working version, tested and committed	
Days 61–90	Ship it; read and contribute to one real open-source project	

BREAK IT

Ask an AI coding assistant to write one function for your project. Review it as you would a stranger's code: read it, test its edge cases, and find at least one thing to change. Note which course skills that review required.

CHECKPOINT

- Every rating has evidence beside it.
- Your plan names a project, a weekly time budget and three dated deliverables.